90.9
Pass

Shivam Jain

PDF generated at: 23 Feb 2026 04:06:33 UTC

View this report on HackerRank

Score

90.9% • 159.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 50 min 51 sec on 23 Feb 2026 08:43:45 IST

Candidate Information

Email	shivamjain17021996@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	23 Feb 2026 08:43:45 IST
Time taken	50 min 51 sec/ 90 min
Work Experience	> 5 years
Invited by	Vipul
Invited on	20 Feb 2026 15:17:04 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	100%
3	Problem Solving Intermediate	78%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	100%	Hard	100%
Spring	100%	Application Properties Annotations	100%
Medium	90%	Pointers	78%
Java	100%	Multithreading	100%
Interfaces	100%	OOP	100%

Questions

Spring Boot (Advanced) • 25 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	3 min 39 sec	✓

2	Cracker Multiple Choice	Spring Boot (Advanced)	5/5	-	4 min 13 sec	✓
3	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	5/5	-	3 min 18 sec	✓
4	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	3 min 1 sec	✓
5	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 13 sec	✓

Problem Solving (Intermediate) • 59 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Quiz Competition Coding	Problem Solving (Intermediate)	59/75	-	21 min 57 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Safe Cracking Coding	Java (Intermediate)	75/75	-	11 min 6 sec	✓

1. Specification

✓ Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● **Specification** ✓
- ○ Criteria
- ○ Query

No comments.

---

2. Cracker

Correct

Multiple Choice | Spring Boot | Hard

Question description

Cracker.java

```
@Component
public class Cracker {
```

```
    public void startCracking(){
        System.out.println("Cracking...");
    }
    public void stopCracking(){
        System.out.println("Cracking Stopped!");
    }
}
```

CrackerService.java

```
@Service
public class CrackerService {

    @Autowired
    Cracker cracker;
    public void workWithNewCracker() {
        cracker.stopCracking();
        cracker = new Cracker();
    }
    public void startWorking() {
        cracker.startCracking();
    }
}
```

CrackerWorker.java

```
@Service
public class CrackerWorker {

    @Autowired
    CrackerService crackerService;

    public void startWorking() {
        crackerService.startWorking();
        crackerService.workWithNewCracker();
        crackerService = new CrackerService();
        crackerService.startWorking();
    }
}
```

Consider these classes are managed in Spring Boot. What happens when 'crackerWorker.startWorking()' is called in the Spring context?

Interviewer guidelines

CrackerService in CrackerWorker will work perfectly until it's replaced with new CrackerService. When that happens it just replace existing injected bean with empty pojo. Which doesn't have it's cracker object so when it's startWorking method is called it will try to execute null cracker attributes startWorking method and NullPointerException will arise.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ "Cracking..." "Cracking Stopped!" will be written to console
- ○ "Cracking..." "Cracking Stopped!" "Cracking..." will be written to console
- ● **"Cracking..." "Cracking Stopped!" NullPointerException will be thrown** ✓
- ○ None of above

No comments.

3. Configuration Properties

Correct

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).
- ○ The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- ● ** A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final. ** ✓
- ○ An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.
- ○ A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

---

4. Spring Scopes

✓ Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope("request") creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope("request").
- [x] @Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

---

5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ private void manage(HttpSecurity http) {   http.csrf().allowed() }
- ○ private void manage(HttpSecurity http) {   http.csrf() }
- ○ private void manage(HttpSecurity http) {   http.csrf().enable() }
- ● ** None of the above. CSRF is configured by default when Spring Security Dependency is added. ** ✓

No comments.

---

6. Quiz Competition

Partially correct

Coding | Medium | Pointers

Question description

A class has students with various talents, each represented by an integer from 1 to *talentsCount*. You need to form teams for a quiz competition, where each team must have at least one member with each talent.

Teams must be formed from consecutive students in the array. For each possible starting position, determine the minimum number of students needed to form a valid team. If it is not possible to form a team with all talents from a particular starting position, return -1 for that position.

Example

talentsCount = 3

talent = [1, 2, 3, 2, 1]

- Starting at position 1: [1, 2, 3] includes all talents, minimum size = 3
- Starting at position 2: [2, 3, 2, 1] is the smallest subarray with all talents, minimum size = 4
- Starting at position 3: [3, 2, 1] includes all talents, minimum size = 3
- Starting at positions 4 and 5: Cannot form a team with all talents, return -1

The result is [3, 4, 3, -1, -1].

Function Description

Complete the function *teamSize* in the editor with the following parameter(s):

int talent[n]: the students' talents

int talentsCount : the number of talents represented numbered from 1 to *talentsCount*

Returns

int[n]: at each index, the minimum size subarray required, or -1 if an appropriate subarray does not exist

Constraints

- $1 \le n, talentsCount \le 10^5$
- $1 \le talent[i] \le talentsCount$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of elements in *talent*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *talent[i]*.

The next line contains the integer *talentCount*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
8       talent[] size n = 8
1       talent = [1, 1, 2, 2, 3, 1, 3, 2]
1
2
2
3
1
3
2
3       talentsCount = 3
```

Sample Output

```
5
4
4
3
4
3
-1
-1
```

Explanation

The shortest subarrays for each position:

Beginning at the first position: [1, 1, 2, 2, 3] (length 5)

Second position: [1, 2, 2, 3] (length 4)

Third position: [2, 2, 3, 1] (length 4)

Fourth position: [2, 3, 1] (length 3)

Fifth position: [3, 1, 3, 2] (length 4)

Sixth position: [1, 3, 2] (length 3)

No further subarrays will have 3 elements.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
5       talent[] size n = 5
1       talent = [1, 1, 1, 1, 1]
1
1
1
1
1       talentsCount = 1
```

Sample Output

```
1
1
1
1
1
```

Explanation

Since there is only 1 type of talent, any 1 person can form a team.

▼ SAMPLE CASE 2

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
9       talent[] size n = 9
7       talent = [7, 5, 3, 4, 6, 1, 7, 2, 4]
5
3
4
6
1
7
2
4
7       talentsCount  7
```

Sample Output

```
8
7
-1
-1
-1
```

```
-1
-1
-1
-1
```

Explanation

There have to be at least *7* elements in any subarray to have one from each talent. The shortest subarrays at each position are:

Beginning at the first position: [7, 5, 3, 4, 6, 1, 7, 2] (length 8)

Second position: [5, 3, 4, 6, 1, 7, 2] (length 7)

No further subarrays will have 7 elements.

Interviewer guidelines

▼ SOLUTION

This can be solved using two-pointer technique. For a particular left index (L), we move the right index (R) forward until we have k distinct elements, then once we get it, we can move L forward. Each element will be traversed at most two times.

Time Complexity: O(n) or O(nLogn) (with set/map).

C++ Solution:

```
int f[100005];

vector<int> teamSize(vector<int> talent, int talentsCount) {
    int k = talentsCount;
    int n = talent.size();
    assert(n>=1&&n<=100000);
    assert(k>=1&&k<=n);
    for(int i=0;i<n;i++)
    {
        assert(talent[i]<=k&&talent[i]>=1);
    }
    int cnt = 0;
    int j = 0;
    vector<int> ans(n);
    for(int i=0;i<n;i++)
```

```
    {
        int x = talent[i];
        f[x]++;
        if(f[x]==1)
            cnt++;
        while(cnt==k&&j<=i)
        {
            ans[j] = i - j + 1;
            int y = talent[j];
            f[y]--;
            if(f[y]==0)
                cnt--;
            j++;
        }
    }
    while(j<n)
        ans[j++] = -1;
    return ans;
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'teamSize' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
```

```
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY talent
     *  2. INTEGER talentsCount
     */

    public static List<Integer> teamSize(List<Integer> talent, int talentsCount) {
    // Write your code here
    int n=talent.size();
    List<Integer> result = new ArrayList<>();

    for(int start =0;start<n;start++){
        int[] freq =new int[talentsCount+1];
        int uniqueCount=0;
        int minSize =-1;
      for(int end=start;end<n;end++){
            int t= talent.get(end);
          if(freq[t]==0){
            uniqueCount++;
          }
        freq[t]++;
        if(uniqueCount== talentsCount){
            minSize = end -start+1;
            break;
        }

    }
    result.add(minSize);
    }
    return result;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int talentCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> talent = IntStream.range(0, talentCount).mapToObj(i -> {
            try {
```

```
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int talentsCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> result = Result.teamSize(talent, talentsCount);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0645 sec	37.6 KB
TestCase 1	Easy	Sample	Success	1	0.0553 sec	37.8 KB
TestCase 2	Easy	Sample	Success	1	0.057 sec	37.5 KB

TestCase 3	Easy	Hidden	Success	3	0.1773 sec	61.7 KB
TestCase 4	Easy	Hidden	Success	3	0.1717 sec	62.4 KB
TestCase 5	Easy	Hidden	Success	3	0.1904 sec	62.1 KB
TestCase 6	Easy	Hidden	Success	8	2.2571 sec	138 KB
TestCase 7	Easy	Hidden	Success	8	2.339 sec	161 KB
TestCase 8	Easy	Hidden	Success	8	2.2693 sec	160 KB
TestCase 9	Easy	Hidden	Success	8	2.2903 sec	144 KB
TestCase 10	Easy	Hidden	Success	8	3.897 sec	330 KB
TestCase 11	Easy	Hidden	Terminated due to timeout	0	4.0375 sec	278 KB
TestCase 12	Easy	Hidden	Terminated due to timeout	0	4.0422 sec	216 KB
TestCase 13	Easy	Hidden	Success	7	3.9504 sec	341 KB

No comments.

7. Java: Safe Cracking

Correct

Coding | Medium | Java | Multithreading | Interfaces | OOP

Question description

Implement a *SafeCracking* class that executes a sequence of operations to rob a bank vault. The class should coordinate with an *IActions* interface to perform the robbery in the correct order.

Your implementation must follow a specific sequence of operations:

1. First, unlock the vault door.
2. Then, simultaneously determine the safe deposit box number and hack its PIN.
3. Finally, open the safe deposit box using the obtained information.

Method to Implement

```
void rob(IActions actions)
```

This method should coordinate with the provided *IActions* object to execute the following steps in order:

1. Call *unlockTheDoor()* to get the door passcode string.
2. Concurrently call both:
   - *hackSecretPin(passcodeOfDoor)* to get the safe deposit box PIN
   - *figureOutSafeDepositBoxNumber(passcodeOfDoor)* to get the safe deposit box number
3. Call *openSafeDepositLock(safeDepositBoxNumber, safetyDepositBoxPin)* to complete the robbery.

Note: The concurrent operations in step 2 are critical for the time-sensitive nature of the robbery scenario.

Example

The system will validate your implementation by checking if the operations are called in the correct order with the proper parameters. A successful implementation will output:

```
true
```

This indicates that all operations were performed correctly, and the robbery was successful. If the operations are performed in the wrong order or with incorrect parameters, the output will be *false*.

Constraints

- *1* ≤ lengths of *passcodeOfDoor*, *safeDespositBoxNumber, safetyDepositBoxPin* ≤ *10*

▼ INPUT FORMAT FOR CUSTOM TESTING

There is no input passed from stdin.

▼ SAMPLE CASE 0

Sample Output

```
true
```

Explanation

- stdout shows *true* if it is a successful robbery otherwise, it shows *false.*

Interviewer guidelines

Java 8 and Java 15 Solution

```
class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{
        ExecutorService threadPool = Executors.newFixedThreadPool(2);
        String passcodeOfDoor = actions.unlockTheDoor();
        Future<String> hackSecretPinTask = threadPool.submit(() -> actions.hackSecretPin(passcodeOf
Door));
        Future<String> figureOutSafeDepositBoxNumberTask = threadPool.submit(() -> actions.figureO
utSafeDepositBoxNumber(passcodeOfDoor));

        String safeDespositBoxNumber = figureOutSafeDepositBoxNumberTask.get();
        String safeDepositBoxPin = hackSecretPinTask.get();

        actions.openSafeDepositLock(safeDespositBoxNumber, safeDepositBoxPin);
        threadPool.shutdown();
    }
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.util.*;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}

import java.io.*;
import java.util.*;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

interface IActions{
    public String unlockTheDoor();
    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException;
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin);
    public boolean getRobberyVerdict();
}


class SafeCracking {
    public void rob(IActions actions) throws InterruptedException, ExecutionException{
```

```
        String passcode = actions.unlockTheDoor();

        CompletableFuture<String> pinFuture = CompletableFuture.supplyAsync(()->{
            try{
                return actions.hackSecretPin(passcode);
            }catch(Exception e){
                throw new RuntimeException(e);

            }
        });

        CompletableFuture<String> boxNumberFuture = CompletableFuture.supplyAsync(()->{
            try{
                return actions.figureOutSafeDepositBoxNumber(passcode);
            }catch(Exception e){
                throw new RuntimeException(e);
            }
        });

        String pin = pinFuture.get();
        String boxNumber = boxNumberFuture.get();

        actions.openSafeDepositLock(boxNumber, pin);
    }
}

public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
```

```java
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

```

```
    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
public class Solution {
    public static void main(String[] args) throws InterruptedException, ExecutionException{
        IActions actions = Actions.actions;
        new SafeCracking().rob(actions);
        System.out.println(actions.getRobberyVerdict());
    }
}

class Actions implements IActions{
    public static IActions actions = new Actions();

    private String passcodeOfDoor;
    private String safetyBoxNumber;
    private String pinOfSafetyBox;

    private String passedPasscodeOfDoor;
    private String passedSafetyBoxNumber;
    private String passedPinOfSafetyBox;

    private Actions() {
        this.passcodeOfDoor = randomStringGenerator(10);
        this.safetyBoxNumber = randomStringGenerator(10);
        this.pinOfSafetyBox = randomStringGenerator(10);
    }

    private String randomStringGenerator(int targetStringLength) {
        int leftLimit = 48; // numeral '0'
        int rightLimit = 122; // letter 'z'
        Random random = new Random();

        String generatedString = random.ints(leftLimit, rightLimit + 1)
            .filter(i -> (i <= 57 || i >= 65) && (i <= 90 || i >= 97))
            .limit(targetStringLength)
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();

        return generatedString;
    }
```

```java

    public boolean getRobberyVerdict(){
        return (passcodeOfDoor == passedPasscodeOfDoor) && (pinOfSafetyBox == passedPinOfSafetyBox)
            && (safetyBoxNumber == passedSafetyBoxNumber);
    }

    public String unlockTheDoor() {
        return this.passcodeOfDoor;
    }

    public String hackSecretPin(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        this.passedPasscodeOfDoor = passcodeOfDoor;
        Thread.sleep(2000);
        return this.pinOfSafetyBox;
    }

    public  String figureOutSafeDepositBoxNumber(final String passcodeOfDoor) throws InterruptedException, ExecutionException {
        Thread.sleep(2000);
        return this.safetyBoxNumber;
    }

    public void openSafeDepositLock(final String safetyDespositBoxNumber, final String safetyDepositBoxPin) {
        this.passedSafetyBoxNumber = safetyDespositBoxNumber;
        this.passedPinOfSafetyBox = safetyDepositBoxPin;
    }
}
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 0	Easy	Sample	Success	1	2.0703 sec	37.7 KB
Testcase 1	Hard	Hidden	Success	74	2.0506 sec	38.1 KB

No comments.